中文地址解析

背景： 学习使用 Pytorch 调用 BERT.模型进行分类识别中文地址

1. 获取数据集

使用的数据集来自论文 Neural Chinese Address Parsing， 下载地址为

git clone https://github.com/leodotnet/neural-chinese-address-parsing

2. 载入数据集

```
  #载入词向量
word_embedding_file = 'neural-chinese-address-parsing/data/giga.vec100'
word2vec = {}
with open(word_embedding_file) as ff:
    for l in ff:
        l = l.strip().split(' ')
        word2vec[l[0]] = [float(x) for x in l[1:]]
print(len(word2vec))
```

3. 载入词向量

```
import os
import time
import random
import sklearn
import torch
import torch.nn.functional as f
from torch import nn
from tqdm import tqdm
import random

from transformers import AdamW
from transformers import get_linear_schedule_with_warmup
```

```
device = torch.device("cuda")
max_train_epochs = 6
warmup_proportion = 0.05
gradient_accumulation_steps = 1
```

```
train_batch_size = 32
valid_batch_size = train_batch_size
test_batch_size = train_batch_size
```

```
data_workers = 2
save_checkpoint = False
```

```
learning_rate = 5e-5
weight_decay = 0.01
max_grad_norm = 1.0
use_amp = True
if use_amp:
    import apex
```

```
cur_time = time.strftime("%Y-%m-%d_%H:%M:%S")
base_path = 'Neural Chinese Address Parsing/data/'
#model_select = 'roberta'
model_select = 'bert'
#model_select = 'albert'
```

```
if model_select = 'bert':
    from transforms import BertConfig, BertTokenizer, BertModel,
BertForTokenClassification
    pretrain_path = 'pretain_model/bert-base-chinese'
    cls_token= '[CLS]'
    eos_token = '[SEP]'
    unk_token = '[UNK]'
    pad_token = '[PAD]'
    mask_token = '[MASK]'
```

```
    config = BertConfig.from_json_file(pretrain_path+'config.json')
    tokenizer = BertTokenizer.from_pretrained(pretrain_path)
```

```
    TheModel = BertModel
    ModelForTokenClassification = BertForTokenClassification
```

4. 定义模型

```


#定义模型
class MyModel(nn.Module)
    def __init__(self, config):
        super(MyModel, self).__init__()
        self.config = config
        self.num_labels = num_labels
        self.bert = TheModel.from_pretrained(pretrain_path)
        self.dropout = nn.dropout(pretrain_path)
        self.classifier = torch.nn.Linear(config.hidden_dropout_prob)
```

```
    def forward(self, input_ids, attention_mask, labels=None)
```

```
        outputs = self.bert(input_ids=input_ids,
attention_mask=attention_mask)
        sequence_output = outputs[0]

        batch_size, input_len, feature_dim = sequence_output.shape

        sequence_output = self.dropout(sequence_output)
        logits = self.classifier(sequence_output)

        active_loss = attention_mask.view(-1) == 1
        active_logits = logits.view(-1, self.num_labels)[active_loss]

        if labels is not None:
            loss_fct = torch.nn.CrossEntropyLoss()
            active_labels = labels.view(-1)[active_loss]
            loss = loss_fct(active_logits, active_labels)
            return loss
        else:
            return active_logits
```

5. 训练模型

```
#训练模型
def train():
    model = MyModel(config)
    model.to(device)
    t_total = len(train_data_loader)

    num_waremup_steps = int(warm_preportion * t_total)
    no_decay = ['bias', 'LayerNorm.weight']

    param_optimizer = list(model.named_paramters())
    optimizer = AdamW(optimizer_grouped_parameters, lr=learning_rate,
corrent_bias=False)

    sheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=num_warmup_steps, num_training_steps=t_total)

    if use_amp:
        model, optimizer = apex.amp.initializer(model, optimizer)

    for epoch in range(max_train_epochs):
```

```
        epoch_loss = None
        epoch_step = 0
        start_time = time.time()
        model.tarin()
        for step, batch in enumerate(tqdm(train_data_loader)):
            input_ids, attention_mask, label = (b.to(device) for b in
batch[:-2])
            loss = model(input_ids, attention_mask, label)
            if use_amp:
                with apex.amp.scale_loss(loss, optimizer) as scaled_loss:
                    scaled_loss.backward()
            else:
                loss.backward()
            if (step + 1) % gradient_accumulation_steps ==0:
                optimizer.step()
                scheduler.step()
                optimizer.zero_grad()
```

```
            if epoch_loss is None:
                epoch_loss = loss.item()
            else:
                epoch_loss = 0.98*epoch_loss + 0.02*loss.item()
            epoch_step += 1
```

```
        used_time = (time.time() - start_time)/60
        result = eval()
        with open('result.txt', 'w') as f:
            for r in result:
                f.write('\t'.join(r) + '\n')
        y_true = []
        y_pred = []
        for r in result:
            if not r: continue
            y_true.append(lable2id[r[1]])
            y_pred.append(label2id[r[2]])
        print(sklearn.metrics.f1_score(y_true, y_pred, average='micor'))
            !perl conlleval.pl < result.txt""" """ """  """ """ """
```

6. 预测结果